

CSE344 – Systems Programming

Homework #1 Report

Fatih Emre OĞAN - 230104004090

1. Overview

myFind is a POSIX-compatible recursive file search program written in C. It searches a given directory tree and prints all matching files in a formatted tree structure. The program supports filtering by filename (with + regex), file size, file type, permissions, and hard link count.

2. How the Problem Was Solved

The program follows these steps to solve the problem:

- Command-line arguments are parsed using getopt(). All filters are stored in a SearchCriteria struct.
- The root directory is opened and the traverse() function is called recursively on every subdirectory.
- For each file, matches_criteria() checks all active filters (filename, size, type, permissions, links).
- Matching files and the directories that lead to them are printed in tree format using | and dashes.
- A SIGINT handler catches CTRL-C and exits cleanly, freeing all resources.

3. Design Decisions

3.1 SearchCriteria Struct

All filter values are stored in a single SearchCriteria struct. Unused fields are set to NULL or -1, which allows easy checking inside matches_criteria(). This keeps the code clean and easy to extend.

3.2 Custom Regular Expression Engine

The regex_match() function implements the + operator from scratch without any external library. The algorithm is recursive: if the next character in the pattern is +, it greedily consumes all matching characters and backtracks to find the best match. Comparison is case-insensitive using tolower().

Example: pattern los+t+file matches LOSSSTTTTFILE because s+ consumes multiple s characters and t+ consumes multiple t characters.

3.3 Tree Output with Lookahead

Only directories that contain or lead to a matching file are printed. This is done by a `subtree_has_match()` function that checks a subtree before printing its root. Every directory is still physically visited to satisfy the full traversal requirement.

3.4 `lstat()` Instead of `stat()`

`lstat()` is used instead of `stat()` so that symbolic links are not followed. This allows correct `-t l` matching on symbolic links.

3.5 Signal Handling

SIGINT (CTRL-C) is caught via `sigaction()`. The handler prints an informational message using the async-signal-safe `write()` call and exits cleanly with `_exit()`.

4. Implemented Requirements

Requirement	Status
-w mandatory path parameter	✓ Implemented
-f filename with + regex (case-insensitive)	✓ Implemented
-b file size in bytes (exact match)	✓ Implemented
-t file type filter (all 7 types)	✓ Implemented
-p permission string (9 characters)	✓ Implemented
-l hard link count	✓ Implemented
At least one criterion required	✓ Implemented
Full recursive subtree traversal	✓ Implemented
Formatted tree output (--- per level)	✓ Implemented
'No file found' message	✓ Implemented
All errors printed to stderr	✓ Implemented
Usage info on bad/missing arguments	✓ Implemented
CTRL-C clean exit with message	✓ Implemented
No memory leaks (valgrind clean)	✓ Implemented
Makefile provided	✓ Implemented
Zero compilation warnings (-Wall)	✓ Implemented

5. Failed Requirements

All requirements specified in the homework document have been successfully implemented. No known failures or missing features exist at the time of submission.

6. Testing

The program was tested on Ubuntu 24.04 LTS. The following test cases were verified:

6.1 Compilation (zero warnings)

Command: `make clean && make`

Expected: Run without errors and warnings

```
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ make clean && make
rm -f myFind
gcc -Wall -Wextra -std=c11 -o myFind myFind.c
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

6.2 Single + Regex Test

Command: `./myFind -w testdir -f "los+file"`

Expected: matches losfile, lossfile, lossfile — does NOT match lostfile

```
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ ./myFind -w testdir -f "los+file"
testdir
|---subdir1
|-----losfile
|-----lossfile
|-----lossfile
|-----lossfile
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

6.3 Double + Regex Test

Command: `./myFind -w testdir -f "los+t+file"`

Expected: matches lostfile and losssttttfile

```
|-----losssttttfile
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ ./myFind -w testdir -f "los+t+file"
testdir
|---subdir1
|-----subdir2
|-----losssttttfile
|-----lostfile
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

6.4 File Type Filter (-t f)

Command: `./myFind -w testdir -t f`

Expected: lists all regular files recursively

```
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ ./myFind -w testdir -t f
testdir
|--hello.txt
|--subdir1
|-----logfile
|-----lossfile
|-----losssfile
|-----subdir2
|-----losssttttfile
|-----lostfile
|--subdir3
|-----world.txt
|-----testfile
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

6.5 File Size Filter (-b 0)

Command: `./myFind -w testdir -b 0`

Expected: lists all files with size exactly 0 bytes

```
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ ./myFind -w testdir -b 0
testdir
|--hello.txt
|--subdir1
|-----logfile
|-----lossfile
|-----losssfile
|-----subdir2
|-----losssttttfile
|-----lostfile
|--subdir3
|-----world.txt
|-----testfile
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

6.6 Permission Filter (-p)

Command: `./myFind -w testdir -p "rw-rw-r--"`

Expected: lists files with exactly those permissions

```
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ ./myFind -w testdir -p "rw-rw-r--"
testdir
|--hello.txt
|--subdir1
|-----logfile
|-----lossfile
|-----losssfile
|-----subdir2
|-----losssttttfile
|-----lostfile
|--subdir3
|-----world.txt
|-----testfile
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

6.7 Combined Filters

Command: `./myFind -w testdir -f "los+file" -t f`

Expected: applies both filename and type filters together

```
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ ./myFind -w testdir -f "los+file" -t f
testdir
|---subdir1
|-----losfile
|-----lossfile
|-----losssfile
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

6.8 Error Cases

Missing criteria, missing `-w`, and invalid directory all produce informative error messages printed to stderr.

```
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ ./myFind -w testdir
Error: at least one search criterion must be specified
Usage: ./myFind -w path [-f name] [-b bytes] [-t type] [-p perms] [-l links]
  -w path    : directory to search recursively (required)
  -f name    : filename pattern ('+' = one or more of preceding char)
  -b bytes   : exact file size in bytes
  -t type    : d=dir f=file l=symlink s=socket b=block c=char p=pipe
  -p perms   : 9-char permission string (e.g. rwxr-xr--)
  -l links   : number of hard links
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

```
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ ./myFind -w /yok -f "test"
Error: stat('/yok'): No such file or directory
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

```
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ ./myFind -f "test"
Error: -w (search path) is required
Usage: ./myFind -w path [-f name] [-b bytes] [-t type] [-p perms] [-l links]
  -w path    : directory to search recursively (required)
  -f name    : filename pattern ('+' = one or more of preceding char)
  -b bytes   : exact file size in bytes
  -t type    : d=dir f=file l=symlink s=socket b=block c=char p=pipe
  -p perms   : 9-char permission string (e.g. rwxr-xr--)
  -l links   : number of hard links
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

6.9 CTRL-C Clean Exit

When the user presses CTRL-C during execution, the program prints 'SIGINT received. Exiting gracefully.' and exits cleanly.

```
|-----Makefile
|-----Kconfig
^C
SIGINT received. Exiting gracefully.
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

6.10 Valgrind Memory Check

valgrind --leak-check=full reports zero memory leaks and zero errors.

```
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$ valgrind --leak-check=full ./myFind -w testdir -f "los+file"
==18046== Memcheck, a memory error detector
==18046== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==18046== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==18046== Command: ./myFind -w testdir -f los+file
==18046==
testdir
|--subdir1
|-----losfile
|-----lossfile
|-----losssfile
==18046==
==18046== HEAP SUMMARY:
==18046==    in use at exit: 0 bytes in 0 blocks
==18046==   total heap usage: 8 allocs, 8 frees, 230,736 bytes allocated
==18046==
==18046== All heap blocks were freed -- no leaks are possible
==18046==
==18046== For lists of detected and suppressed errors, rerun with: -s
==18046== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
fatih@fatih:~/Desktop/CSE344_Systems_Programming/solved/HW1$
```

7 References

- [1] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, San Francisco, CA, 2010. ISBN: 978-1-59327-220-3.
- [2] M. Kerrisk, *The Linux Programming Interface*, Chapter 15 — File Attributes: stat() and lstat() system calls for retrieving file metadata and permission bits
- [3] M. Kerrisk, *The Linux Programming Interface*, Chapter 18 — Directories and Links: opendir(), readdir(), and closedir() for directory traversal.
- [4] M. Kerrisk, *The Linux Programming Interface*, Chapter 20 — Signals: Fundamental Concepts: sigaction() for reliable signal handling and SIGINT behavior.
- [5] IEEE and The Open Group, *The Open Group Base Specifications Issue 7*, 2018 Edition (POSIX.1-2017).
Available: <https://pubs.opengroup.org/onlinepubs/9699919799/>